FM
FLOWMATRIX.DEV

Premium Workflow

A practical guide – step by step

Social Listening & Sentiment Spike Pipeline

Collect brand mentions from social channels, analyze sentiment with an LLM, cluster emerging topics, persist trends in Supabase, and alert the team when sentiment spikes.

MARKETING & ANALYTICS | ADVANCED | 45–60 MINUTES

Created & Verified on: 2026-11-04 08:01 UTC

Premium Kit €34.99

OVERVIEW

Social Listening & Sentiment Spike Pipeline

Collect brand mentions from social channels, analyze sentiment with an LLM, cluster emerging topics, persist trends in Supabase, and alert the team when sentiment spikes.

CATEGORY: Marketing & Analytics

DIFFICULTY: Advanced

DURATION: 45–60 minutes

WHAT YOU USE IT FOR

Detect brand sentiment shifts and emerging topics in real time.

TOOLS IN THIS WORKFLOW

- X API
- OpenAI API
- Supabase
- Slack

Created & Verified on: 2026-11-04 08:01 UTC

Verification & Link Validity Notice: This workflow blueprint was generated and technically verified on 2026-11-04 08:01 UTC. At the exact time of creation, all referenced domains, API endpoints, and external resource links were tested and confirmed reachable. Because third-party APIs and web services evolve continuously, domains, endpoints, or links inside this document may change or become unavailable over time.

BEFORE YOU START

What you need

- An X API account and access token
- An OpenAI API key
- A Supabase project and service-role key
- A Slack incoming webhook URL
- Python 3.10 or newer (standard library only) and a terminal

GLOSSARY

Advanced concepts

Bearer Token Authentication

A method for securing API requests using a token that grants access.

Rate-Limit Backoff

A strategy for handling API rate limits by waiting before retrying requests.

PostgREST Upsert

An operation that inserts or updates a record in a database based on its existence.

Z-Score Spike Detection

A statistical method for identifying outliers in data based on standard deviations.

STEP 1 OF 5

Fetch brand mentions from X

TECHNICAL DETAILS

This step acts as the initial data collector, fetching brand mentions from the X API. It utilizes the endpoint `/2/tweets/search/recent` with a GET request, requiring authorization via a Bearer token. Edge cases include handling rate limits, which may necessitate implementing a backoff strategy, and managing pagination if the response contains more results than can be returned in a single call.

WHAT YOU NEED TO DO

- Set up an X API account and obtain an access token.
- Define the search query for brand mentions.
- Implement error handling for API responses.

CODE SNIPPET

```
import json
import urllib.request
import urllib.parse

def fetch_mentions():
url = 'https://api.twitter.com/2/tweets/search/recent'
query = {'query': 'your_brand_name'}
headers = {'Authorization': 'Bearer YOUR_X_API_TOKEN'}
request_url = f'{url}?{urllib.parse.urlencode(query)}'
req = urllib.request.Request(request_url, headers=headers)
with urllib.request.urlopen(req) as response:
data = json.loads(response.read())
return [item['text'] for item in data['data']]

mentions = fetch_mentions()
```

CHECK AT THE END

- [ ] Verify that the API returns a list of mentions.
- [ ] Ensure that the mentions are correctly extracted from the response.

RESULT

Recent brand mentions are successfully fetched from the X API.

LINKS

X API Search Docs: https://developer.twitter.com/en/docs/twitter-api/tweets/search/introduction?ref=flowmatrix

STEP 2 OF 5

Analyze sentiment with OpenAI

TECHNICAL DETAILS

This step processes the fetched mentions by sending them to the OpenAI API for sentiment analysis. The endpoint used is `/v1/chat/completions`, requiring a POST request with a structured prompt. Edge cases include handling API response errors and ensuring that the sentiment analysis returns valid float values for classification.

WHAT YOU NEED TO DO

- Set up an OpenAI API account and obtain an API key.
- Define the prompt structure for sentiment analysis.
- Implement error handling for API responses.

CODE SNIPPET

```
import json
import urllib.request

def analyze_sentiment(mentions):
url = 'https://api.openai.com/v1/chat/completions'
headers = {'Authorization': 'Bearer YOUR_OPENAI_API_KEY', 'Content-Type': 'application/json'}
records = []
for mention in mentions:
prompt = f'Analyze the sentiment of the following text: {mention}'
data = json.dumps({'model': 'gpt-4o-mini', 'messages': [{'role': 'user', 'content': prompt}]})
req = urllib.request.Request(url, data.encode('utf-8'), headers)
with urllib.request.urlopen(req) as response:
content = json.loads(response.read())['choices'][0]['message']['content']
sentiment = float(content)
records.append({'text': mention, 'sentiment': sentiment})
return records

analyzed = analyze_sentiment(mentions)
```

CHECK AT THE END

- [ ] Verify that sentiment values are returned as floats.
- [ ] Ensure that each mention is associated with its sentiment.

RESULT

Sentiment analysis results are generated for each brand mention.

LINKS

OpenAI API Documentation: https://platform.openai.com/docs/api-reference/chat/create?ref=flowmatrix

STEP 3 OF 5

Cluster emerging topics

TECHNICAL DETAILS

This step analyzes the sentiment records to cluster mentions based on keyword frequency. It uses a simple frequency count to identify common themes. Edge cases include handling mentions with no common keywords and ensuring that the clustering logic can handle varying lengths of input data.

WHAT YOU NEED TO DO

• Define the logic for clustering based on keyword frequency.

• Implement data structures to store keyword counts.

• Test the clustering logic with sample data.

CODE SNIPPET

```
from collections import Counter

def cluster_topics(analyzed):
keywords = Counter()
for record in analyzed:
words = record['text'].split()
keywords.update(words)
return keywords.most_common(10)

clusters = cluster_topics(analyzed)
```

CHECK AT THE END

- [ ] Verify that the most common keywords are identified.
- [ ] Ensure that the clustering logic handles edge cases.

RESULT

Emerging topics are successfully clustered based on keyword frequency.

LINKS

Python Collections Documentation: https://docs.python.org/3/library/collections.html?ref=flowmatrix

STEP 4 OF 5

Persist trends in Supabase

TECHNICAL DETAILS

This step sends the clustered trends to Supabase for persistent storage. It uses the PostgREST API for upserting records into a specified table. Edge cases include handling API errors and ensuring that the data structure matches the Supabase table schema.

WHAT YOU NEED TO DO

- Set up a Supabase project and create a table for trends.
- Define the data structure for upserting records.
- Implement error handling for API responses.

CODE SNIPPET

```
import json
import urllib.request


def persist_trends(clusters):
url = 'https://YOUR_SUPABASE_URL/rest/v1/your_table_name'
headers = {
'apikey': 'YOUR_SUPABASE_KEY',
'Authorization': 'Bearer YOUR_SUPABASE_KEY',
'Content-Type': 'application/json',
'Prefer': 'return=representation'
}
for keyword, count in clusters:
data = json.dumps({'keyword': keyword, 'count': count})
req = urllib.request.Request(url, data.encode('utf-8'), headers)
with urllib.request.urlopen(req) as response:
if response.status not in (200, 201, 204):
raise Exception('Failed to upsert data')

persist_trends(clusters)
```

CHECK AT THE END

- [ ] Verify that the trends are correctly upserted into Supabase.
- [ ] Ensure that the data structure matches the Supabase table schema.

RESULT

Trends are successfully persisted in Supabase.

LINKS

Supabase API Documentation: https://supabase.com/docs/reference/javascript/overview?ref=flowmatrix

STEP 5 OF 5

Alert on sentiment spikes

TECHNICAL DETAILS

This step analyzes the sentiment data to compute z-scores for detecting spikes. If a spike exceeds a defined threshold, an alert is sent to Slack via an incoming webhook. Edge cases include handling cases where no spikes are detected and ensuring that the alert logic is triggered only under specific conditions.

WHAT YOU NEED TO DO

• Define the logic for calculating z-scores.

• Set a threshold for detecting sentiment spikes.

• Implement the alert mechanism to notify the team.

CODE SNIPPET

```
import json
import urllib.request
import statistics


def alert_on_spikes(analyzed):
sentiments = [record['sentiment'] for record in analyzed]
mean = statistics.mean(sentiments)
stdev = statistics.stdev(sentiments)
threshold = 2.0 # Define z-score threshold for spikes
for sentiment in sentiments:
z_score = (sentiment - mean) / stdev
if z_score >= threshold:
send_alert(sentiment)


def send_alert(sentiment):
url = 'YOUR_SLACK_WEBHOOK_URL'
data = json.dumps({'text': f'Sentiment spike detected: {sentiment}'})
req = urllib.request.Request(url, data.encode('utf-8'), headers={'Content-Type': 'application/json'})
with urllib.request.urlopen(req) as response:
if response.status != 200:
raise Exception('Failed to send alert')

alert_on_spikes(analyzed)
```

CHECK AT THE END

- [ ] Verify that alerts are sent correctly to Slack.
- [ ] Ensure that the z-score calculation is accurate.

RESULT

Alerts are successfully sent to Slack upon detecting sentiment spikes.

LINKS

Slack Incoming Webhooks Documentation: https://api.slack.com/messaging/webhooks?ref=flowmatrix

PRINTABLE CHECKLIST

Your complete checklist

1. Fetch brand mentions from X

- [ ] Verify that the API returns a list of mentions.
- [ ] Ensure that the mentions are correctly extracted from the response.

2. Analyze sentiment with OpenAI

- [ ] Verify that sentiment values are returned as floats.
- [ ] Ensure that each mention is associated with its sentiment.

3. Cluster emerging topics

- [ ] Verify that the most common keywords are identified.
- [ ] Ensure that the clustering logic handles edge cases.

4. Persist trends in Supabase

- [ ] Verify that the trends are correctly upserted into Supabase.
- [ ] Ensure that the data structure matches the Supabase table schema.

5. Alert on sentiment spikes

- [ ] Verify that alerts are sent correctly to Slack.
- [ ] Ensure that the z-score calculation is accurate.

TROUBLESHOOTING

If something goes wrong

Problem: Rate limit exceeded error from X API

Fix: Implement exponential backoff before retrying the request.

Problem: Invalid API key error from OpenAI

Fix: Verify the API key and ensure it is correctly set in the environment.

Problem: Failed to upsert data in Supabase

Fix: Check the Supabase API key and ensure the table schema matches the data structure.

RECOMMENDED TOOLS & RESOURCES

X API: https://developer.twitter.com/en/docs/twitter-api

OpenAI API: https://platform.openai.com/?ref=flowmatrix

Supabase: https://supabase.com/?ref=flowmatrix

Slack: https://slack.com/

Verification & Link Validity Notice: This workflow blueprint was generated and technically verified on 2026-11-04 08:01 UTC. At the exact time of creation, all referenced domains, API endpoints, and external resource links were tested and confirmed reachable. Because third-party APIs and web services evolve continuously, domains, endpoints, or links inside this document may change or become unavailable over time.